

Q1. What is Python?

Answer:

Python is a high-level, interpreted, general-purpose programming language. Being a general-purpose language, it can be used to build almost any type of application with the right tools/libraries. Additionally, python supports objects, modules, threads, exception-handling, and automatic memory management which help in modelling real-world problems and building applications to solve these problems.

Q2. What are the benefits of using Python?

Answer:

- * Python is a general-purpose programming language that has a simple, easy-to-learn syntax that emphasizes readability and therefore reduces the cost of program maintenance. Moreover, the language is capable of scripting, is completely open-source, and supports third-party packages encouraging modularity and code reuse.
- * Its high-level data structures, combined with dynamic typing and dynamic binding, attract a huge community of developers for Rapid Application Development and deployment.

Q3. What is a dynamically typed language?

Answer:

Typing refers to type-checking in programming languages. In a strongly-typed language, such as Python, "1" + 2 will result in a type error since these languages don't allow for "type-coercion" (implicit conversion of data types). On the other hand, a weakly-typed language, such as Javascript, will simply output "12" as result.

Type-checking can be done at two stages -

- * Static - Data Types are checked before execution.
- * Dynamic - Data Types are checked during execution.

Python is an interpreted language, executes each statement line by line and thus type-checking is done on the fly, during execution. Hence, Python is a Dynamically Typed Language.

Python is an interpreted language, executes each statement line by line and thus type-checking is done on the fly, during execution. Hence, Python is a Dynamically Typed Language.

Q4. What is an Interpreted language?

Answer:

An Interpreted language executes its statements line by line. Languages such as Python, Javascript, R, PHP, and Ruby are prime examples of Interpreted languages. Programs written in an interpreted language runs directly from the source code, with no intermediary compilation step.

Q5. What is PEP 8 and why is it important?

Answer:

PEP stands for Python Enhancement Proposal. A PEP is an official design document providing information to the Python community, or describing a new feature for Python or its processes. PEP 8 is especially important since it documents the style guidelines for Python Code. Apparently contributing to the Python open-source community requires you to follow these style guidelines sincerely and strictly.

Q6. What is Scope in Python?

Answer:

A scope is a block of code where an object in Python remains relevant. Namespaces uniquely identify all the objects inside a program. However, these namespaces also have a scope defined for them where you could use their objects without any prefix.

Q7. What are the different type of scope in python?

Answer:

- * A local scope refers to the local objects available in the current function.
- * A global scope refers to the objects available throughout the code execution since their inception.

- * A module-level scope refers to the global objects of the current module accessible in the program.
- * An outermost scope refers to all the built-in names callable in the program. The objects in this scope are searched last to find the name referenced.

Note: Local scope objects can be synced with global scope objects using keywords such as `global`.

Q8. What are lists and tuples?

Answer:

Lists and Tuples are both sequence data types that can store a collection of objects in Python. The objects stored in both sequences can have different data types. Lists are represented with square brackets `['sara', 6, 0.19]`, while tuples are represented with parentheses `('ansh', 5, 0.97)`.

But what is the real difference between the two? The key difference between the two is that while lists are mutable, tuples on the other hand are immutable objects. This means that lists can be modified, appended or sliced on the go but tuples remain constant and cannot be modified in any manner.

Q9. What are the common built-in data types in Python?

Answer:

Python doesn't require data types to be defined explicitly during variable declarations. If the knowledge of data types and their compatibility with each other are neglected, Python provides `type()` and `isinstance()` functions to check the type of these variables.

These data types can be grouped into the following categories-

- * **None Type:** `None` keyword represents the null values in Python. Boolean equality operation can be performed using these `NoneType` objects.

- * **Numeric Types:** There are three distinct numeric types - integers, floating-point numbers, and complex numbers. Additionally, booleans are a sub-type of integers.

- * **Sequence Types:** According to Python Docs, there are three basic Sequence Types - lists, tuples, and range objects. Sequence types have the `in` and `not in` operators defined for their traversing their elements. These operators share the

ts. Sequence types have the `in` and `not in` operators defined for their traversing their elements. These operators share the same priority as the comparison operations.

*** Mapping Types:**

A mapping object can map hashable values to random objects in Python. Mappings objects are mutable and there is currently only one standard mapping type, the dictionary.

*** Set Types:** Currently, Python has two built-in set types - set and frozenset. set type is mutable and supports methods like `add()` and `remove()`. frozenset type is immutable and can't be modified after creation.

*** Modules:** Module is an additional built-in type supported by the Python Interpreter. It supports one special operation, i.e., attribute access: `mymod.myobj`, where `mymod` is a module and `myobj` references a name defined in `m`'s symbol table. The module's symbol table resides in a very special attribute of the module `_dict_`, but direct assignment to this module is neither possible nor recommended.

*** Callable Types:** Callable types are the types to which function call can be applied. They can be user-defined functions, instance methods, generator functions, and some other built-in functions, methods and classes.

Q10. What is pass in Python?

Answer:

The `pass` keyword represents a null operation in Python. It is generally used for the purpose of filling up empty blocks of code which may execute during runtime but has yet to be written. Without the `pass` statement in the following code, we may run into some errors during code execution.

Q11. What are modules and packages in Python?

Answer:

Modules, are simply Python files with a `.py` extension and can have a set of functions, classes, or variables defined and implemented. They can be imported and initialized once using the `import` statement. If partial functionality is needed, import the requisite classes or functions using `from foo import bar`.

Packages allow for hierarchical structuring of the module namespace using dot notation. As, modules help avoid clashes between global variable names, in a similar manner, packages help avoid clashes between module names. Creating a package is easy since it makes use of the system's inherent file structure. So just stuff the modules into a folder and there you have it, the folder name as the package name. Importing a module or its contents from this package requires the package name as prefix to the module name joined by a dot.

Q12. What are global, protected and private attributes in Python?

Answer:

- * Global variables are public variables that are defined in the global scope. To use the variable in the global scope inside a function, we use the `global` keyword.
- * Protected attributes are attributes defined with an underscore prefixed to their identifier eg. `_sara`. They can still be accessed and modified from outside the class they are defined in but a responsible developer should refrain from doing so.
- * Private attributes are attributes with double underscore prefixed to their identifier eg. `__ansh`. They cannot be accessed or modified from the outside directly and will result in an `AttributeError` if such an attempt is made.

Q13. What is the use of `self` in Python?

Answer:

`Self` is used to represent the instance of the class. With this keyword, you can access the attributes and methods of the class in python. It binds the attributes with the given arguments. `self` is used in different places and often thought to be a keyword. But unlike in C++, `self` is not a keyword in Python.

Q14. What is `__init__`?

Answer:

`__init__` is a constructor method in Python and is automatically called to allocate memory when a new object/instance is created. All classes have a `__init__` method associated with them. It helps in distinguishing methods and attributes of a class from local variables.

Q15. What is break, continue and pass in Python?

Answer:

Break- The break statement terminates the loop immediately and the control flows to the statement after the body of the loop.

Continue- The continue statement terminates the current iteration of the statement, skips the rest of the code in the current iteration and the control flows to the next iteration of the loop.

Pass- the pass keyword in Python is generally used to fill up empty blocks and is similar to an empty statement represented by a semi-colon in languages such as Java, C++, Javascript, etc.

Q16. What are unit tests in Python?

Answer:

- * Unit test is a unit testing framework of Python.

- * Unit testing means testing different components of software separately. Can you think about why unit testing is important? Imagine a scenario, you are building software that uses three components namely A, B, and C. Now, suppose your software breaks at a point time. How will you find which component was responsible for breaking the software? Maybe it was component A that failed, which in turn failed component B, and this actually failed the software. There can be many such combinations.

- * This is why it is necessary to test each and every component properly so that we know which component might be highly responsible for the failure of the software.

Q17. What is docstring in Python?

Answer:

- * Documentation string or docstring is a multiline string used to document a specific code segment.

- * The docstring should describe what the function or method does.

Q18. What is slicing in Python?

Answer:

- * As the name suggests, 'slicing' is taking parts of.
- * Syntax for slicing is [start : stop : step]
- * start is the starting index from where to slice a list or tuple
- * stop is the ending index or where to stop.
- * step is the number of steps to jump.
- * Default value for start is 0, stop is number of items, step is 1.
- * Slicing can be done on strings, arrays, lists, and tuples.

Example:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
print(numbers[1 : : 2]) #output : [2, 4, 6, 8, 10]
```

Q19. Explain how can you make a Python Script executable on Unix?

Answer:

Script file must begin with `#!/usr/bin/env python`

Q20. What is the difference between Python Arrays and lists?

Answer:

- * Arrays in python can only contain elements of same data types i.e., data type of array should be homogeneous. It is a thin wrapper around C language arrays and consumes far less memory than lists.
- * Lists in python can contain elements of different data types i.e., data type of lists can be heterogeneous. It has the disadvantage of consuming large memory.

Q21. How is memory managed in Python?

Answer:

- * Memory management in Python is handled by the Python Memory Manager. The memory allocated by the manager is in form of a private heap space dedicated to Python. All Python objects are stored in this heap and being private, it is inaccessible to the programmer. Though, python does provide some core API functions to work upon the private heap space.
- * Additionally, Python has an in-built garbage collection to recycle the unused memory for the private heap space.

Q22. What are Python namespaces? Why are they used?

Answer:

A namespace in Python ensures that object names in a program are unique and can be used without any conflict. Python implements these namespaces as dictionaries with 'name as key' mapped to a corresponding 'object as value'. This allows for multiple namespaces to use the same name and map it to a separate object. A few examples of namespaces are as follows:

- * Local Namespace includes local names inside a function. the namespace is temporarily created for a function call and gets cleared when the function returns.
 - * Global Namespace includes names from various imported packages/ modules that are being used in the current project. This namespace is created when the package is imported in the script and lasts until the execution of the script.
 - * Built-in Namespace includes built-in functions of core Python and built-in names for various types of exceptions.
- The lifecycle of a namespace depends upon the scope of objects they are mapped to. If the scope of an object ends, the lifecycle of that namespace comes to an end. Hence, it isn't possible to access inner namespace objects from an outer namespace.

Q23. What is Scope Resolution in Python?

Answer:

Sometimes objects within the same scope have the same name but function differently. In such cases, scope resolution comes into play in Python automatically.

Example:

* Python modules namely 'math' and 'cmath' have a lot of functions that are common to both of them - log10(), acos(), exp() etc. To resolve this ambiguity, it is necessary to prefix them with their respective module, like math.exp() and cmath.exp().

* Consider the code below, an object temp has been initialized to 10 globally and then to 20 on function call. However, the function call didn't change the value of the temp globally. Here, we can observe that Python draws a clear line between global and local variables, treating their namespaces as separate identities.

```
temp = 10 # global-scope variable
```

```
def func():
```

```
    temp = 20 # local-scope variable
```

```
    print(temp)
```

```
print(temp) # output => 10
```

```
func() # output => 20
```

```
print(temp) # output => 10
```

This behavior can be overridden using the global keyword inside the function, as shown in the following example:

```
temp = 10 # global-scope variable
```

```
def func():
```

```
    global temp
```

```
    temp = 20 # local-scope variable
```

```
    print(temp)
```

```
print(temp) # output => 10
```

```
func() # output => 20
```

```
print(temp) # output => 20
```

Q24. What are decorators in Python?

Answer:

Decorators in Python are essentially functions that add functionality to an existing function in Python without changing the structure of the function itself. They are represented the @decorator_name in Python and are called in a bottom-up fashion. The beauty of the decorators lies in the fact that besides adding functionality to the output of the method, they can even accept arguments for functions and can further modify those arguments before passing it to the function itself. The inner nested function, i.e. 'wrapper' function, plays a significant role here. It is implemented to enforce encapsulation and thus, keep itself hidden from the global scope.

Q25. What are Dict and List comprehensions?

Answer:

Python comprehensions, like decorators, are syntactic sugar constructs that help build altered and filtered lists, dictionaries, or sets from a given list, dictionary, or set. Using comprehensions saves a lot of time and code that might be considerably more verbose (containing more lines of code).

Q26. What is lambda in Python?

Answer:

Lambda is an anonymous function in Python, that can accept any number of arguments, but can only have a single expression. It is generally used in situations requiring an anonymous function for a short time period.

Q27. How do you copy an object in Python?

Answer:

In Python, the assignment statement (= operator) does not copy objects. Instead, it creates a binding between the existing object and the target variable name. To create copies of an object in Python, we need to use the copy module. Moreover, there are two ways of creating copies for the given object using the copy module -

Shallow Copy is a bit-wise copy of an object. The copied object created has an exact copy of the values in the original object. If either of the values is a reference to other objects, just the reference addresses for the same are copied.

Deep Copy copies all values recursively from source to target object, i.e. it even duplicates the objects referenced by the source object.

the source object.

Q28. What is the difference between xrange and range in Python?

Answer:

xrange() and range() are quite similar in terms of functionality. They both generate a sequence of integers, with the only difference that range() returns a Python list, whereas, xrange() returns an xrange object.

Q29. What is pickling and unpickling?

Answer:

Pickling:

- * Pickling is the name of the serialization process in Python. Any object in Python can be serialized into a byte stream and dumped as a file in the memory. The process of pickling is compact but pickle objects can be compressed further. Moreover, pickle keeps track of the objects it has serialized and the serialization is portable across versions.

- * The function used for the above process is pickle.dump().

Unpickling:

- * Unpickling is the complete inverse of pickling. It deserializes the byte stream to recreate the objects stored in the file and loads the object to memory.

- * The function used for the above process is pickle.load().

Note: Python has another, more primitive, serialization module called marshall, which exists primarily to support .pyc files in Python and differs significantly from the pickle.

Q30. What are generators in Python?

Answer:

Generators are functions that return an iterable collection of items, one at a time, in a set manner. Generators, in general, are used to create iterators with a different approach. They employ the use of yield keyword rather than return to return a generator object.

Q31. What is PYTHONPATH in Python?

Answer:

PYTHONPATH is an environment variable which you can set to add additional directories where Python will look for modules and packages. This is especially useful in maintaining Python libraries that you do not wish to install in the global default location.

Q32. What is the use of help() and dir() functions?

Answer:

help() function in Python is used to display the documentation of modules, classes, functions, keywords, etc. If no parameter is passed to the help() function, then an interactive help utility is launched on the console. dir() function tries to return a valid list of attributes and methods of the object it is called upon. It behaves differently with different objects, as it aims to produce the most relevant data, rather than the complete information.

- * For Modules/Library objects, it returns a list of all attributes, contained in that module.
- * For Class Objects, it returns a list of all valid attributes and base attributes.
- * With no arguments passed, it returns a list of attributes in the current scope.

I

Q33. What is the difference between .py and .pyc files?

Answer:

- * .py files contain the source code of a program. Whereas, .pyc file contains the bytecode of your program. We get bytecode after compilation of .py file (source code). .pyc files are not created for all the files that you run. It is only created for the files that you import.
- * Before executing a python program python interpreter checks for the compiled files. If the file is present, the virtual machine executes it. If not found, it checks for .py file. If found, compiles it to .pyc file and then python virtual machine executes it.
- * Having .pyc file saves you the compilation time.

Q34. How Python is interpreted?

Answer:

- * Python as a language is not interpreted or compiled. Interpreted or compiled is the property of the implementation. Python is a bytecode(set of interpreter readable instructions) interpreted generally.
- * Source code is a file with .py extension.
- * Python compiles the source code to a set of instructions for a virtual machine. The Python interpreter is an implementation of that virtual machine. This intermediate format is called "bytecode".
- * .py source code is first compiled to give .pyc which is bytecode. This bytecode can be then interpreted by the official CPython or JIT(Just in Time compiler) compiled by PyPy.

Q35. How are arguments passed by value or by reference in python?

Answer:

- * Pass by value: Copy of the actual object is passed. Changing the value of the copy of the object will not change the value of the original object.
- * Pass by reference: Reference to the actual object is passed. Changing the value of the new object will change the value of the original object.

Q36. What are iterators in Python?

Answer:

- * An iterator is an object.
- * It remembers its state i.e., where it is during iteration (see code below to see how)
- * `__iter__()` method initializes an iterator.
- * It has a `__next__()` method which returns the next item in iteration and points to the next element. Upon reaching the end of iterable object `__next__()` must return `StopIteration` exception.
- * It is also self-iterable.
- * Iterators are objects with which we can iterate over iterable objects like lists, strings, etc.

Q37. Explain how to delete a file in Python?

Answer:

Use command `os.remove(file_name)`

Example:

```
import os
os.remove("ChangedFile.csv")
print("File Removed!")
```

Q38. Explain `split()` and `join()` functions in Python?

Answer:

- * You can use `split()` function to split a string based on a delimiter to a list of strings.
- * You can use `join()` function to join a list of strings based on a delimiter to give a single string.

Q39. What does `*args` and `**kwargs` mean?

Answer:

`*args`

- * `*args` is a special syntax used in the function definition to pass variable-length arguments.
- * `“*”` means variable length and `“args”` the name used by convention. You can use any other.

`**kwargs`

- * `**kwargs` is a special syntax used in the function definition to pass variable-length keyworded arguments.
- * Here, also, `“kwargs”` is used just by convention. You can use any other name.
- * Keyworded argument means a variable that has a name when passed to a function.
- * It is actually a dictionary of the variable names and its value.

Q40. What are negative indexes and why are they used?

Answer:

- * Negative indexes are the indexes from the end of the list or tuple or string.
- * `Arr[-1]` means the last element of array `Arr[]`

Q41. Is it possible to call parent class without its instance creation?

Answer:

Yes, it is possible if the base class is instantiated by other child classes or if the base class is a static method.

Q42. How is an empty class created in python?

Answer:

An empty class does not have any members defined in it. It is created by using the `pass` keyword (the `pass` command does nothing in python). We can create objects for this class outside the class.

Q43. Differentiate between `new` and `override` modifiers.

Answer:

The `new` modifier is used to instruct the compiler to use the new implementation and not the base class function. The `Override` modifier is useful for overriding a base class function inside the child class.

Q44. Why is `finalize` used? I

Answer:

`Finalize` method is used for freeing up the unmanaged resources and clean up before the garbage collection method is invoked. This helps in performing memory management tasks.

Q45. What is `init` method in python?

Answer:

The `init` method works similarly to the constructors in Java. The method is run as soon as an object is instantiated. It is useful for initializing any attributes or default behaviour of the object at the time of instantiation.

Q46. How will you check if a class is a child of another class?

Answer:

This is done by using a method called `issubclass()` provided by python. The method tells us if any class is a child of another class by returning true or false accordingly.

Q47. What do you understand by reindexing in pandas?

Answer:

Reindexing is the process of conforming a dataframe to a new index with optional filling logic. If the values are missing in the previous index, then NaN/NA is placed in the location. A new object is returned unless a new index is produced that is equivalent to the current one. The copy value is set to False. This is also used for changing the index of rows and columns in the dataframe.

Q48. Can you easily check if all characters in the given string is alphanumeric?

Answer:

This can be easily done by making use of the `isalnum()` method that returns true in case the string has only alphanumeric characters.

Another way is to use `match()` method from the `re (regex)` module as shown:

Q49. Define GIL.

Answer:

GIL stands for Global Interpreter Lock. This is a mutex used for limiting access to python objects and aids in effective thread synchronization by avoiding deadlocks. GIL helps in achieving multitasking (and not parallel computing).

Q50. What is main function in python? How do you invoke it?

Answer:

In the world of programming languages, the main is considered as an entry point of execution for a program. But in python, it is known that the interpreter serially interprets the file line-by-line. This means that python does not provide a main() function explicitly. But this doesn't mean that we cannot simulate the execution of main. This can be done by defining user-defined main() function and by using the `_name_` property of python file. This `_name_` variable is a special built-in variable that points to the name of the current module. This can be done as shown below:

```
def main():  
    print("Hi Interviewbit!")  
if _name=="main":  
    main()
```

DIY CODING QUESTION IN PYTHON

1. Write python function which takes a variable number of arguments.
2. WAP (Write a program) which takes a sequence of numbers and check if all numbers are unique.
3. Write a program for counting the number of every character of a given text file.
4. Write a program to check and return the pairs of a given array A whose sum value is equal to a target value N.
5. Write a Program to add two integers >0 without using the plus operator.
6. Write a Program to solve the given equation assuming that a,b,c,m,n,o are constants:
 $ax + by = c.$ $mx + ny = o$
7. Write a Program to match a string that has the letter 'a' followed by 4 to 8 'b's.
8. Write a Program to convert date from yyyy-mm-dd format to dd-mm-yyyy format.
9. Write a Program to combine two different dictionaries. While combining, if you find the same keys, you can add the values of these same keys. Output the new dictionary
10. How will you access the dataset of a publicly shared spreadsheet in CSV format stored in Google Drive?

11. Convert a given string to int using a single line of code.

We can convert a given string to an integer using a built-in function `int()`. e.g.-

```
a = '5'
```

```
print(int(a))
```

Variable 'a' is a string that is now converted to an integer, as shown below:

Output:

5

12. Write a code snippet to convert a string to a list.

Below is the code to convert string to list in Python.

```
str1 = "Analytics Vidhya"
```

```
print(str1.split(" "))
```

The `split()` function separates the given string by the defined delimiter i.e., `space(" ")` here. Therefore, Analytics and Vidhya break down to two strings in a list.

Output:

```
['Analytics', 'Vidhya']
```

13. How to Check a Number is Palindrome in Python.

14. How To Convert Lists To Strings In Python?

15. How to implement Merge Sort in Python?

16. How to Implement a Linked List in Python?
 17. How to implement Python program to check Leap Year?
 18. How to reverse a number in Python?
 19. How to Find Prime Numbers in Python
 20. How To Implement GCD In Python?
-

TEST YOURSELF

Q1. Suppose `list1 = [3,4,5,2,1,0]`, what is `list1` after `list1.pop(1)`?

- A. `list1 = [3,4,5,2,1]`
- B. `list1 = [3,4,5,2,0]`
- C. `list1 = [3,5,2,1,0]`
- D. `list1 = [3,4,5,2]`

I

Q2. What is the output of the following statement `"Hello World"[::-1]`?

- A. `"Hello World"`
- B. `"World Hello"`

t is the output of the following statement "Hello World"

d Hello"

olleH"

dlroW"

t is the difference between lists and tuples?

Hello"
 olleH"
 dlroW"
 is the difference between lists and

Q6. Which of the following is the function responsible for pickling?

- A. `pickle.save()`
- B. `pickle.store()`
- C. `pickle.pickle()`
- D. `pickle.dump()`

Q7. Which of the following is a protected attribute?

- A. `_sara_`
- B. `__ansh`
- C. `sara`
- D. `ansh__`

Q8. Which of the following is untrue for Python namespaces?

- A. Python namespaces are implemented as a dictionary in Python.
- B. Python namespaces have keys as addresses of the objects.
- C. Lifecycle of a namespace depends upon the scope of the objects they are mapped to.

Q8. Which of the following is untrue for Python namespaces?

- A. Python namespaces are implemented as a dictionary in Python.
- B. Python namespaces have keys as addresses of the objects.
- C. Lifecycle of a namespace depends upon the scope of the objects they are mapped to.
- D. Namespaces ensure that object names in a program are unique.

Q9. Let `list1 = ['s', 'r', 'a', 's']` and `list2 = ['a', 'a', 'n', 'h']`, what is the output of `["".join([i, j]) for i, j in zip(list1, list2)]`?

- A. `['s', 'a', 'r', 'a', 'a', 'n', 's', 'h']`
- B. `['s', 'r', 'a', 's', 'a', 'a', 'n', 'h']`
- C. `['sa', 'ra', 'an', 'sh']`

I
D. `['sa', 'sa', 'sn', 'sh', 'ra', 'ra', 'rn', 'rh', 'aa', 'aa', 'an', 'ah', 'sa', 'sa', 'sn', 'sh']`

Q10. `time.time()` in Python returns?

- A. Current time.

Q10. `time.time()` in Python returns?

- A. Current time.
 - B. Current time in milliseconds.
 - C. Current time in milliseconds since midnight, January 1, 1970.
 - D. Current time in milliseconds since midnight, January 1, 1970 GMT (the Unix time).
-

Answers:

1. `list1 = [3,5,2,1,0]`
2. `"dlroW olleH"`
3. Lists are mutable but tuples are immutable.
4. `1e+20`
5. `__init__` is called manually on object creation.
6. `pickle.dump()`
7. sara
8. Python namespaces have keys as addresses of the objects.
9. `['sa', 'ra', 'an', 'sh']`
10. Current time in milliseconds since midnight, January 1, 1970 GMT (the Unix time).